Cell-wise Storage and BXYZE Conversion in SFEM

October 13, 2026

1 Introduction

The direction of this work is inspired by the main idea behind `FlexibleCG.pdf`: reduce the cost of repeated matrix-free operator applications by changing the data layout used by the high-order element kernels. Instead of applying the operator directly from a unique global vector at every iteration, we introduce a persistent cell-wise representation.

In the standard finite element representation, each global degree of freedom is stored once. Adjacent elements refer to the same global value through an element-local to global map. This representation is compact and natural for assembled algebra, but a matrix-free tensor-product kernel sees indirect memory accesses every time it gathers element data.

The cell-wise representation duplicates shared degrees of freedom per element. Each element owns a dense local tensor of values. This increases memory usage, but makes the data consumed by the volume kernel regular, contiguous, and amenable to blocking and vectorization. After an element-wise operator application, duplicated values must be synchronized by a direct stiffness summation (DSS) step. The present note documents the first step implemented in SFEM: conversion from the global SFEM vector representation to a blocked cell-wise layout called BXYZE.

2 Memory Layout: Global to BXYZE

2.1 SFEM semi-structured connectivity

For a semi-structured hexahedral mesh of level p, each macro element has

$$N = p + 1$$

nodes per coordinate direction and therefore

$$n_{\text{loc}} = N^3$$

local scalar degrees of freedom.

SFEM stores the element connectivity in structure-of-arrays form:

$$\text{elements}[\ell][e] = g.$$

Here e is the element index, ℓ is the local node index, and g is the corresponding global node id. For tensor-product hexahedra we use the local ordering

$$\ell(i_x, i_y, i_z) = i_z N^2 + i_y N + i_x,$$

with

$$0 \le i_x, i_y, i_z < N.$$

Thus i_x is the fastest local coordinate.

The global scalar vector is denoted by

$$u_g[g].$$

The goal of the conversion is to construct a cell-wise vector u_c such that each element-local copy satisfies

$$u_c(e, i_x, i_y, i_z) = u_g\left[\text{elements}[\ell(i_x, i_y, i_z)][e]\right].$$

2.2 Element blocks

The BXYZE layout groups elements into fixed-size blocks of B elements. This B is a data-layout block size, not an SFEM mesh block. For an element e, we define

$$e_b = \left\lfloor \frac{e}{B} \right\rfloor, \qquad b = e \bmod B.$$

The index e_b identifies the element block, while b identifies the lane inside that block.

For example, if $B = 4$, then

e	0	1	2	3	4	5
e_b	0	0	0	0	1	1
b	0	1	2	3	0	1

The last block may be partially filled. Missing lanes are padding lanes and are set to zero by the conversion.

2.3 BXYZE linearization

The BXYZE linear index used in the current SFEM prototype is

$$I(e_b, i_z, i_y, i_x, b) = b + B\left(i_x + N\left(i_y + N\left(i_z + N e_b\right)\right)\right).$$

Equivalently, the strides are

index	stride
b	1
i_x	B
i_y	BN
i_z	BN^2
e_b	BN^3

Thus, for fixed (e_b, i_x, i_y, i_z), the B element lanes are contiguous in memory:

$$u_c[I+0],\ u_c[I+1],\ \ldots,\ u_c[I+B-1].$$

This is the key property of the layout. The volume kernel can operate on the same local tensor point across several elements with contiguous memory accesses in the element-lane dimension.

2.4 Conversion algorithm

Let

$$n_{\text{blocks}} = \left\lceil \frac{n_{\text{elements}}}{B} \right\rceil.$$

The allocated number of scalar values is

$$n_{\text{values}} = n_{\text{blocks}}\, B\, N^3.$$

The conversion iterates over element blocks and local tensor coordinates:

$$\text{for } e_b = 0, \ldots, n_{\text{blocks}} - 1,$$
$$\text{for } i_z, i_y, i_x = 0, \ldots, N - 1.$$

For the current block,

$$e_{\text{begin}} = e_b B, \qquad r = n_{\text{elements}} - e_{\text{begin}},$$

and the number of valid lanes is

$$b_{\text{count}} = \min(r, B).$$

Then, for each valid lane $b < b_{\text{count}}$, the element index is

$$e = e_{\text{begin}} + b,$$

and the conversion performs

$$u_c[I(e_b, i_z, i_y, i_x, b)] = u_g\left[\text{elements}[\ell(i_x, i_y, i_z)][e]\right].$$

For padded lanes $b_{\text{count}} \le b < B$, the output is set to zero.

In code, the inner access pattern is intentionally simple:

$$\text{gid} = \text{elements}[\ell][e_{\text{begin}} + b], \qquad \text{dst}[b] = u_g[\text{gid}].$$

Since SFEM stores $\text{elements}[\ell][e]$ contiguously in e, the connectivity row is read contiguously over b. The output $\text{dst}[b]$ is also contiguous over b. The only remaining indirect access is the read from the global vector $u_g[\text{gid}]$, which is paid once when building the persistent cell-wise representation.

2.5 Current implementation scope

The current implementation is scalar-field, CPU-side, and targets semi-structured hexahedral meshes with explicit p-level tensor-product indexing. The public conversion entry points are implemented in `operators/dss/sfem_dss_bxyze.cpp` and declared in `operators/dss/sfem_dss_bxyze.hpp`. The current code provides single-precision and double-precision variants:

`sfem_dss_global_to_bxyze_real32`, `sfem_dss_global_to_bxyze_real64`.

The next steps are to define the DSS operation on this cell-wise storage and to combine volume operator application plus DSS into a full matrix-free apply path.